

Aufgabe 1: Programmiersprachen

- (a) Diskutieren Sie den Unterschied zwischen kompilierten und interpretierten Programmiersprachen.

Erläuterung:

- **Kompiliert:** Der Quellcode wird vor der Ausführung vollständig in Maschinsprache übersetzt (z. B. C, C++). Dadurch läuft das Programm schnell, ist aber oft an ein bestimmtes Betriebssystem oder eine Architektur gebunden.
- **Interpretiert:** Der Quellcode wird Zeile für Zeile oder Anweisung für Anweisung zur Laufzeit vom Interpreter ausgeführt (z. B. Python, JavaScript). Das erleichtert Testen und Entwicklung, aber die Ausführung ist langsamer, da die Übersetzung ständig stattfindet.

- (b) Nennen Sie je einen Vorteil und einen Nachteil einer kompilierten bzw. einer interpretierten Sprache.

Typ	Vorteil	Nachteil
Kompiliert	Sehr effizient und schnell	Plattformabhängig, längere Übersetzungszeit
Interpretiert	Einfach zu testen und zu debuggen, plattformunabhängig	Langsamere Ausführung, benötigt Interpreter zur Laufzeit

Aufgabe 2: Datentypen

- (a) Nennen Sie den Datentyp jeder Variable im folgenden Code-Ausschnitt.

Listing 1: Code-Ausschnitt zur Bestimmung der Datentypen

```
1 var variable1 = 10 / 3;  
2 var variable2 = "10";  
3 var variable3 = 15f;  
4 var variable4 = "15" + 10;  
5 var variable5 = 15.0;  
6 var variable6 = 15.0d;  
7 var variable7 = variable1 + variable4;  
8 var variable8 = variable5 + variable6;  
9 var variable9 = (int)(double) variable5;  
10 var variable10 = variable3 <= variable5;
```

Variable	Datentyp	Erklärung
variable1	int	10 / 3 → Ganzzahlige Division → Ergebnis 3
variable2	String	In Anführungszeichen → Text
variable3	float	15f → f kennzeichnet Literal als float
variable4	String	"15"+ 10 → String-Verkettung → "1510"
variable5	double	15.0 → Fließkommazahl, standardmäßig double
variable6	double	15.0d → explizit als double markiert
variable7	String	variable1 + variable4 → int + String → automatische Umwandlung zu String
variable8	double	variable5 + variable6 → double + double → Ergebnis double
variable9	int	(int)(double) variable5 → zuerst double-Cast, dann int-Cast → int
variable10	boolean	variable3 <= variable5 → Vergleich → true oder false

(b) Widening Conversions

Widening Conversions (Java Primitive Types)

Definition: Eine Widening Conversion ist eine automatische Typkonvertierung in Java, bei der ein kleinerer primitiver Datentyp in einen größeren Typ konvertiert wird, ohne Datenverlust.

Graphische Darstellung:



Erläuterung: - Kleine Datentypen in Java werden automatisch in größere Typen konvertiert.

- Beispiel: `int -> double` ist gültig; `double -> int` ist nicht automatisch möglich.
- Transitiv abgeleitete Konvertierungen (z. B. `byte -> int`) müssen nicht extra gezeichnet werden.

Aufgabe 3: Boolesche Logik

(a) Was ist die Ausgabe der Methode example1?

```
1 public static void example1() {  
2     boolean a = true;  
3     boolean b = false;  
4     int x = 10;  
5     int y = 20;  
6     int z = 30;  
7  
8     if (a || b) {  
9         System.out.println("C1");  
10  
11         if (x < y && x < z) {  
12             System.out.println("C2");  
13         } else if (x < y || x < z) {  
14             System.out.println("C3");  
15         } else {  
16             System.out.println("C4");  
17         }  
18     }  
19 }
```

Schritt-für-Schritt Analyse:

- Bedingung $a \ || \ b \Rightarrow \text{true} \ || \ \text{false} \Rightarrow \text{true} \Rightarrow$ Ausgabe: C1
- Bedingung $x < y \ \ x < z \Rightarrow 10 < 20 \ \ 10 < 30 \Rightarrow \text{true} \Rightarrow$ Ausgabe: C2

Ausgabe:

C1

C2

(b) Was ist die Ausgabe der Methode example2?

```

1 public static void example2() {
2     int x = 10;
3     int y = 20;
4     int z = 30;
5     int i = 0;
6
7     while (i < 10) {
8         if (x / 10 > i || i * x > z) {
9             System.out.println("L1");
10            i++;
11        } else if (i > y / 3) {
12            System.out.println("L2");
13            i += 2;
14        } else {
15            System.out.println("L3");
16            i += 3;
17        }
18
19        if (i % 2 == 0) {
20            System.out.println("Even: " + i);
21            i++;
22        }
23    }
24 }

```

Schritt-für-Schritt Analyse:

i	Bedingung	Ausgabe	Neuer i
0	$10/10 > 0 \Rightarrow 1 > 0$ true	L1	1
1	$1 > 1$ false, $1 \cdot 10 > 30$ false $\Rightarrow$ else	L3, Even:4	5
5	$1 > 5$ false, $5 \cdot 10 > 30$ true	L1, Even:6	7
7	$1 > 7$ false, $7 \cdot 10 > 30$ true	L1, Even:8	9
9	$1 > 9$ false, $9 \cdot 10 > 30$ true	L1	10 (Ende)

Ausgabe:

L1
 L3
 Even: 4
 L1
 Even: 6
 L1
 Even: 8
 L1

(c) Geben Sie die Wahrheitstabellen für die folgenden booleschen Ausdrücke.

1. $(\neg A \vee B) \wedge (C \vee \neg D) \wedge (C \vee B)$

A	B	C	D	Ergebnis
F	F	F	F	F
F	F	F	T	F
F	F	T	F	T
F	F	T	T	T
F	T	F	F	T
F	T	F	T	T
F	T	T	F	T
F	T	T	T	T
T	F	F	F	F
...	...	...	...	...

2. $\neg(A \vee B \vee C \vee D)$

A	B	C	D	Ergebnis
F	F	F	F	T
else combinations				F

3. $\neg A \wedge \neg B \wedge \neg C \wedge \neg D$

A	B	C	D	Ergebnis
F	F	F	F	T
else combinations				F